

Data Structures

Lecture 16: Linked List

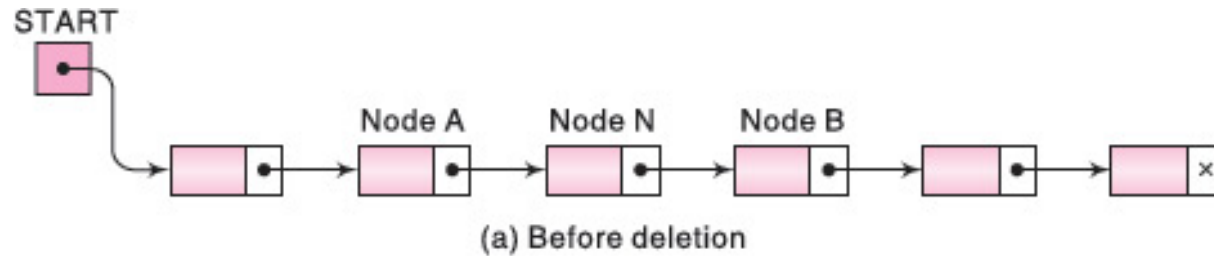
Content



- **Deletion**

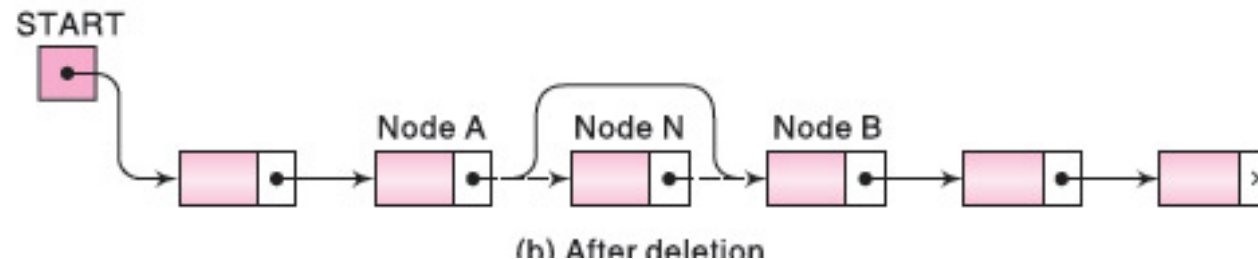
Linked List: Deletion Operation

Let LIST be a linked list with a node N between nodes A and B.



Suppose node N is to be deleted from the linked list.

The deletion occurs as soon as the *next-pointer* field of node A is changed so that it points to node B.



Garbage Collection

Whenever a node is deleted, some memory space becomes reusable.

One way to use this memory is to immediately insert the free space into availability list.
Is this efficient method ?

Another method is used which is called ‘Garbage Collection’.

It works in two steps

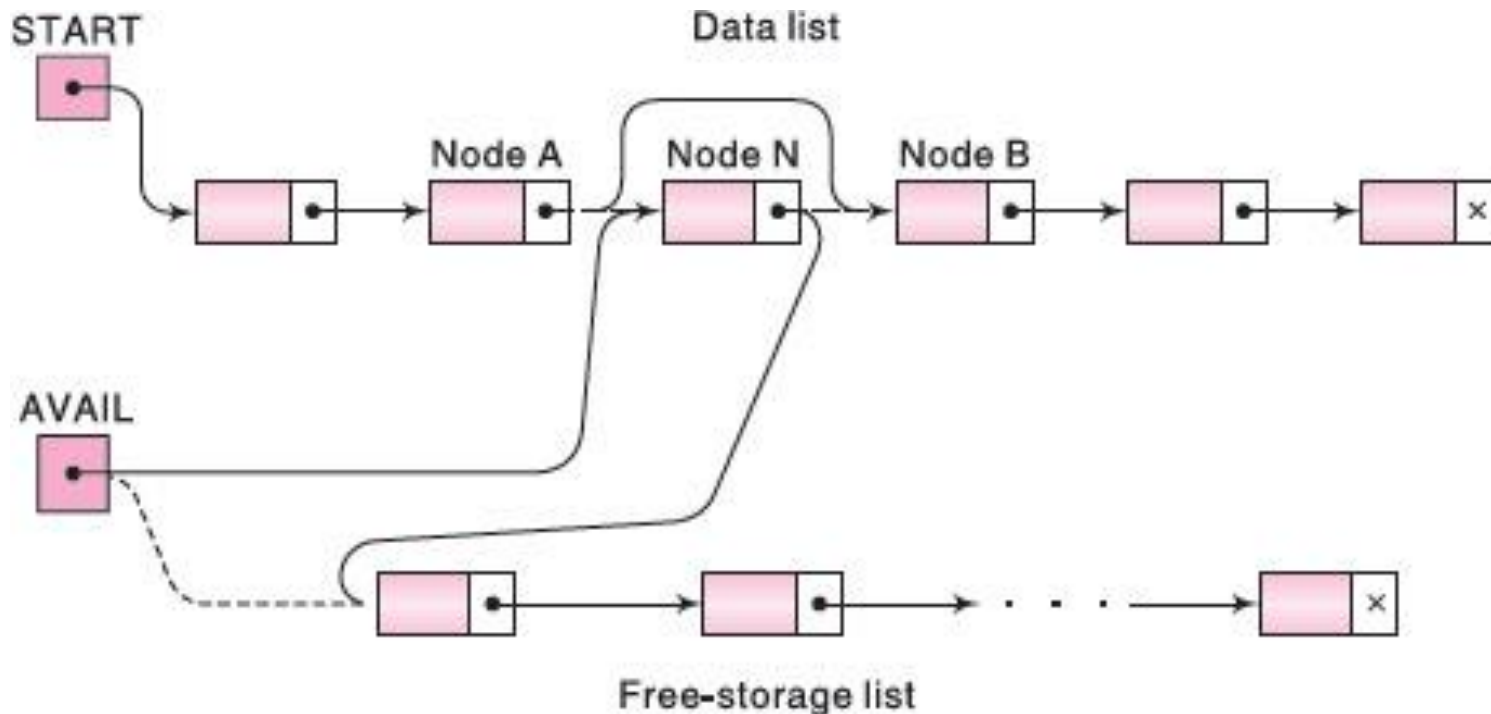
- In first step, the OS goes through all the lists and tags all those cells which are currently being used.
- In the second step, the OS goes through all the lists again and collects untagged space and adds this collected space to availability list.

Linked List: Deletion Operation

The *next-pointer* field of node A now points to node B, where node N previously pointed.

The nextpointer field of N now points to the original first node in the free pool, where AVAIL previously pointed.

AVAIL now points to the deleted node N.



Linked List: Delete Operation

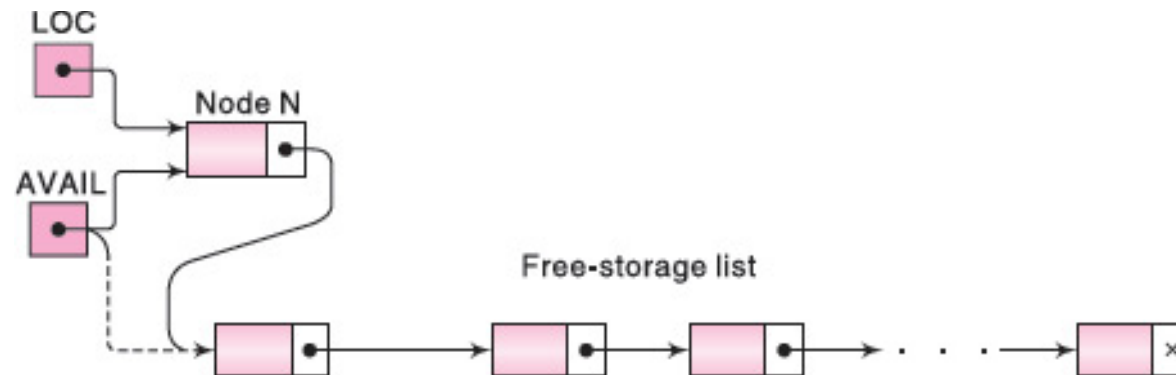
Deletion in a linked list involves removing a new node from the list. There are several types of deletion based on the position from where the new node is to be deleted. It is broadly covered under two cases

- Deletes the node following a given node,
- Deletes the node with a given ITEM of information

Linked List: Delete Operation

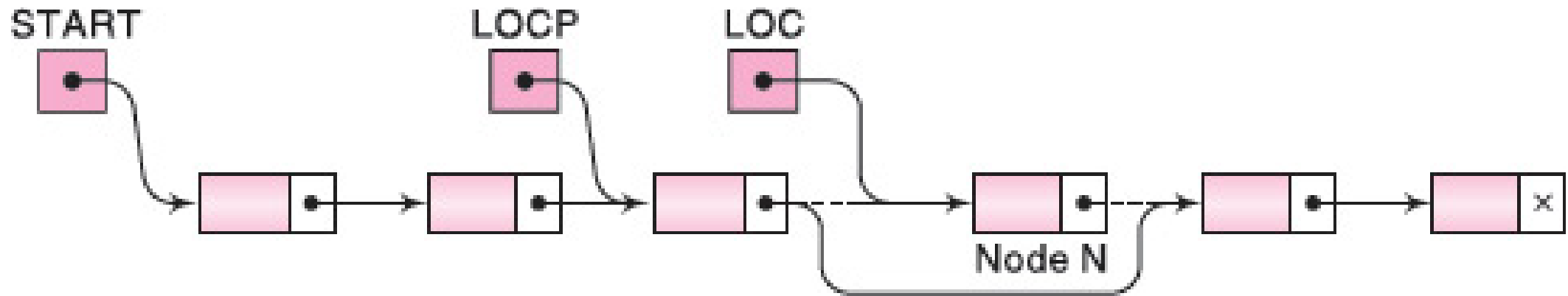
Common Step: Any of the case mentioned in previous slide will consider to delete data in linked list by appending the deleted node in the AVAIL list, So the common step required for different cases are as follows

- Check to see if there is a node in the list. If not, i.e., if $START = NULL$, then the algorithm will print the message UNDERFLOW.
- Return the memory space of the deleted node N to the beginning of the AVAIL list.



Deleting the Node Following a Given Node

Let LIST be a linked list in memory. Suppose we are given the location LOC of a node N in LIST. Furthermore, suppose we are given the location LOCP of the node preceding N or, when N is the first node, we are given $LOCP = \text{NULL}$.



Deleting the Node Following a Given Node

Algorithm

DEL(INFO, LINK, START, AVAIL, LOC, LOCP)

1. If LOCP = NULL
 then: Set START := LINK[START], and Return.
2. Else
 Set LINK[LOCP] := LINK[LOC] //Delete node N
3. Set LINK[LOC] := AVAIL and AVAIL := LOC //Return deleted node
4. EXIT

Deleting the Node with a Given ITEM of Information

Suppose we are given an ITEM of information and we want to delete from the LIST the first node N which contains ITEM.

Before we can delete N from the list, we need to know the location of the node preceding N

- If N is the first node, we set $LOCP = NULL$, and
- If ITEM does not appear in LIST, we set $LOC = NULL$.

Deleting the Node with a Given ITEM of Information

Algorithm

FINDB(INFO, LINK, START, ITEM, LOC, LOCP)

- 1 If START = NULL // List is empty
 then: Set LOC := NULL, LOCP := NULL and Return.
- 2.If INFO[START] := ITEM
 then: Set LOC := START, LOCP= NULL and Return.
- 3.ELSE
4. Set SAVE := START and PTR := LINK[START].
5. Repeat Steps 6 and 7 while PTR ≠ NULL.
6. If INFO[PTR] = ITEM
 then Set LOC := PTR and LOCP := SAVE, and Return
7. Else
 Set SAVE := PTR and PTR := LINK[PTR].
8. SET LOC= NULL and Return.

Use *Deleting the Node Following a Given Node* to delete the node

Linked List: Concatenate Operation

Given two linked lists. The task is to concatenate the second list to the end of the first list.

Approach:

The idea is very simple, first find the last node of the first linked list. Then, update the next pointer of the last node to the head node the second linked list.

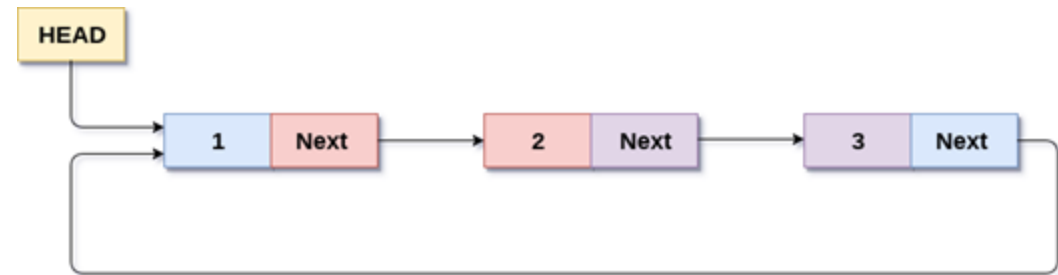
Linked List: Concatenate Operation

Algorithm:

```
1. If START_1=NULL
    return START_2
2. Else
    PTR= START_1
3. While (LINK (PTR)≠ NULL)
    PTR= LINK (PTR)
4. LINK (PTR)= START _2
5. Return START _1
```

Circular Linked List

A circular linked list is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node.



This structure allows for continuous traversal without any interruptions.

Basic Operations

Following are the basic operations supported by a circular linked list.

Insertion

- Adds an element at the beginning of the list.

Deletion

- Deletes an element at the beginning of the list.

Display

- Displays the complete list.

Search

- Searches an element using the given key.

Delete

- Deletes an element using the given key.